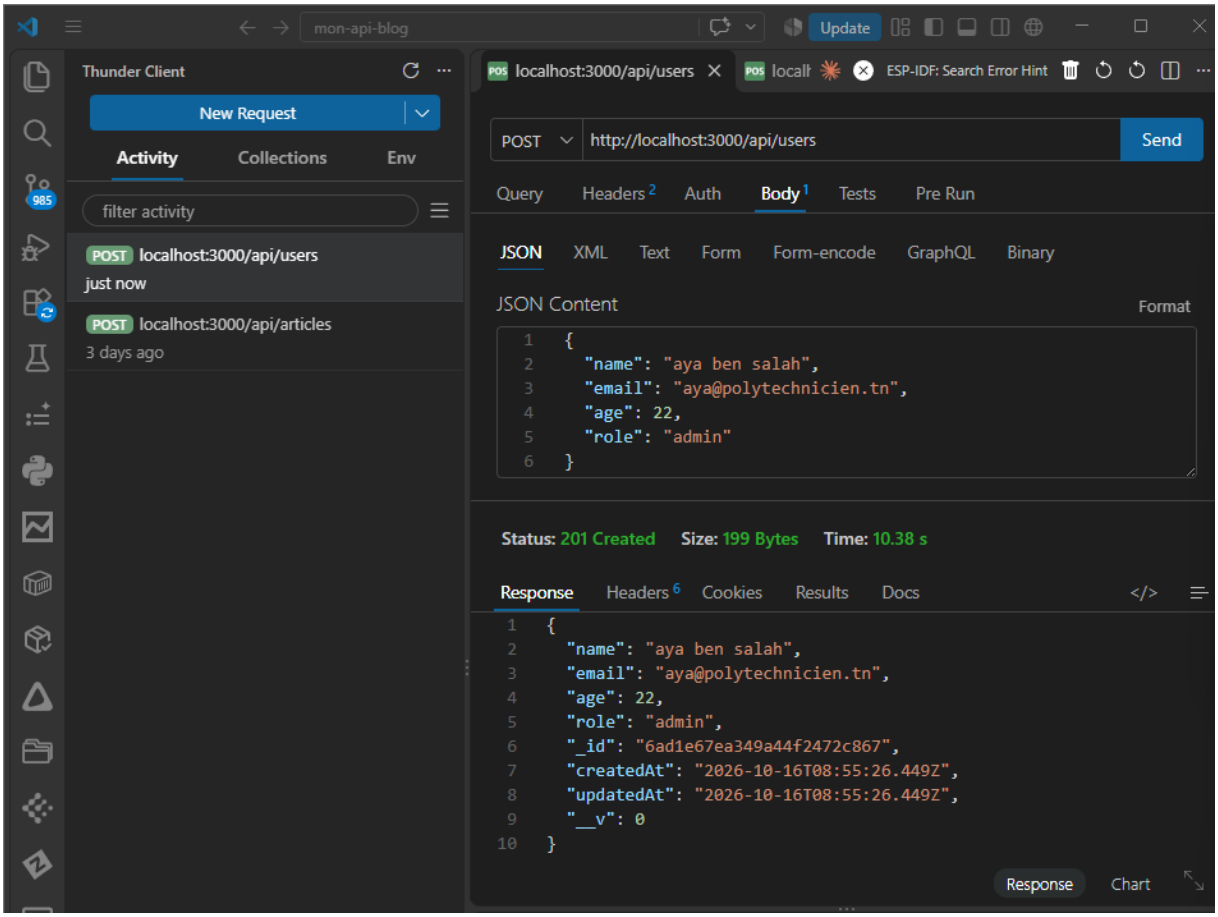
COMPTE-RENDU

MERN Stack

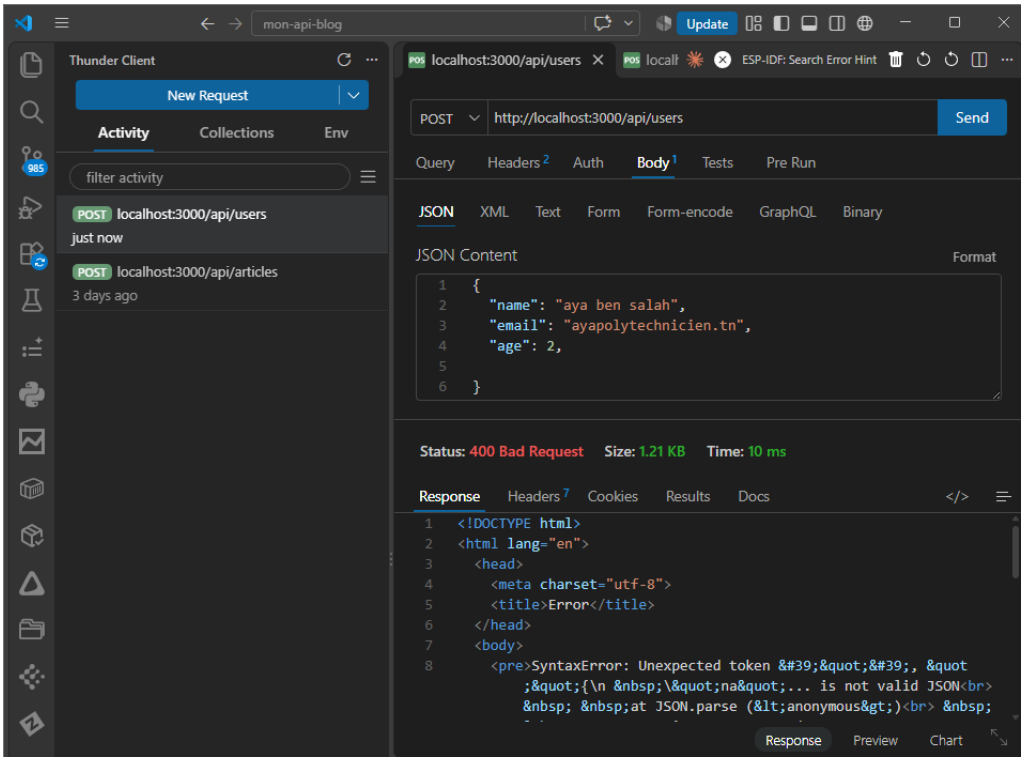
Nom : Adem Boukadi Class : 5éme info DS IA G1

Etap 7 :

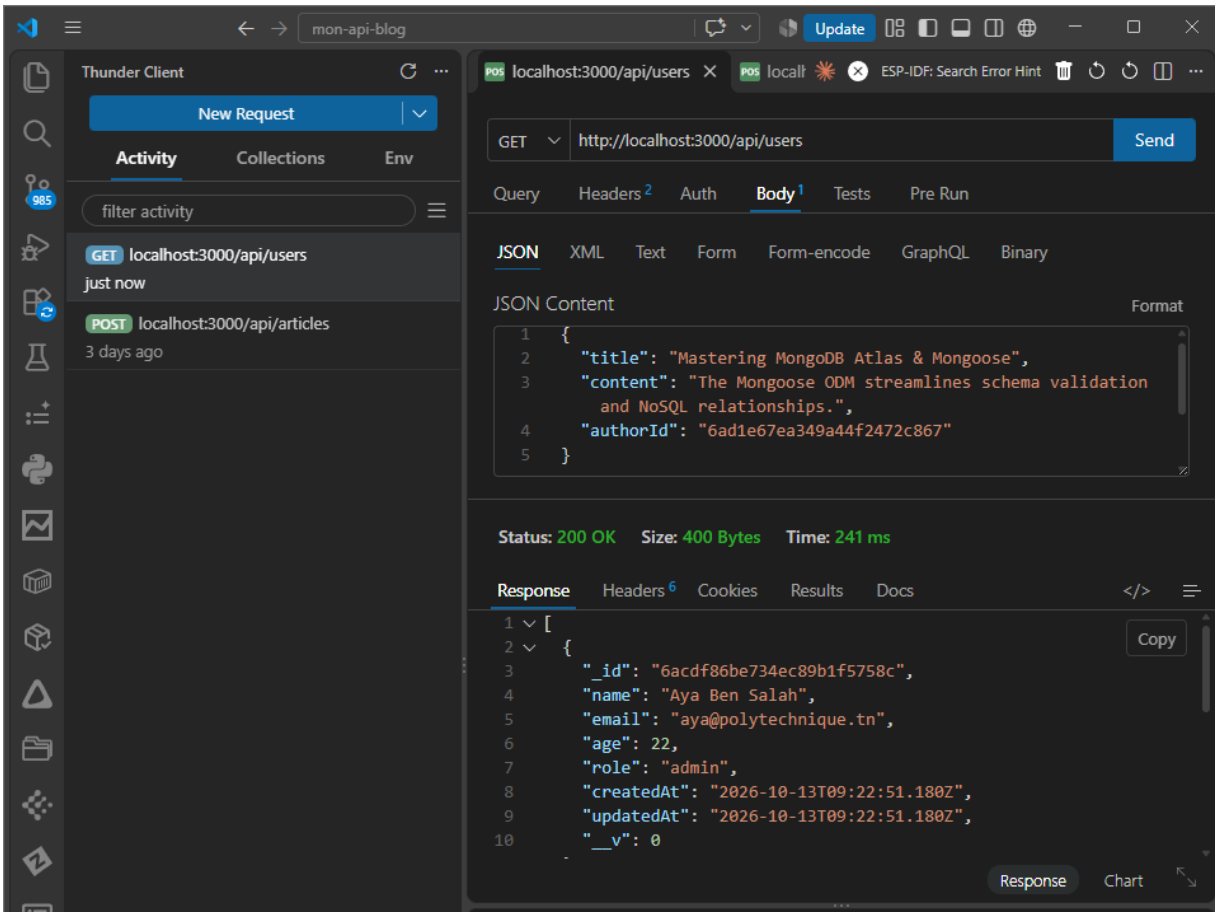
1. Create a valid user (POST http://localhost:3000/api/users):



2. Test Schema constraint validation (Regex & minimum age rejection):



3. Create a linked article (POST http://localhost:3000/api/articles):



Exercice 1:

1. Enrich src/models/userModel.js with two new fields:

• bio: Type String, optional, with a maximum length limit maxlength: [200, 'Bio cannot exceed 200 characters'];

```
bio: {
  type: String,
  maxlength: [200, 'Bio cannot exceed 200 characters']
},
```

• phone: Type String, validated by a regular expression enforcing an 8-digit phone number (e.g., match: [/^\d{8}$/, '8-digit phone number required']).

```
phone: {
  type: String,
  match: [/^\d{8}$/, '8-digit phone number required']
}
```

2. Implement secure user deletion (DELETE /api/users/:id) in controllers/userController.js and declare the corresponding route in routes/userRoutes.js.
3. Referential Integrity Guard: Before deleting a user, check for dependent articles using Article.findOne({ author: req.params.id }). If the user authored at least one published article, deny deletion by returning 400 Bad Request with: "Impossible to delete an author

with published articles".

```
const deleteUser = async (req, res) => {
  try {
    const userId = req.params.id;

    // Vérification de l'intégrité référentielle
    const hasArticles = await Article.findOne({ author: userId });
    if (hasArticles) {
      return res.status(400).send('Impossible to delete an author
    }

    // Suppression si aucun article n'est trouvé
    const user = await User.findByIdAndDelete(userId);
    if (!user) {
      return res.status(404).json({
        message: `Impossible de supprimer : utilisateur ${userId}
      });
    }

    return res.status(200).json({ message: 'Utilisateur supprimé
  } catch (err) {
    return res.status(500).json({ message: err.message });
  }
```

Exercise 2: Professional Handling of Invalid IDs

1. In your controller functions accepting an ID parameter (getArticleById, getUserById), inspect the catch (err) block.

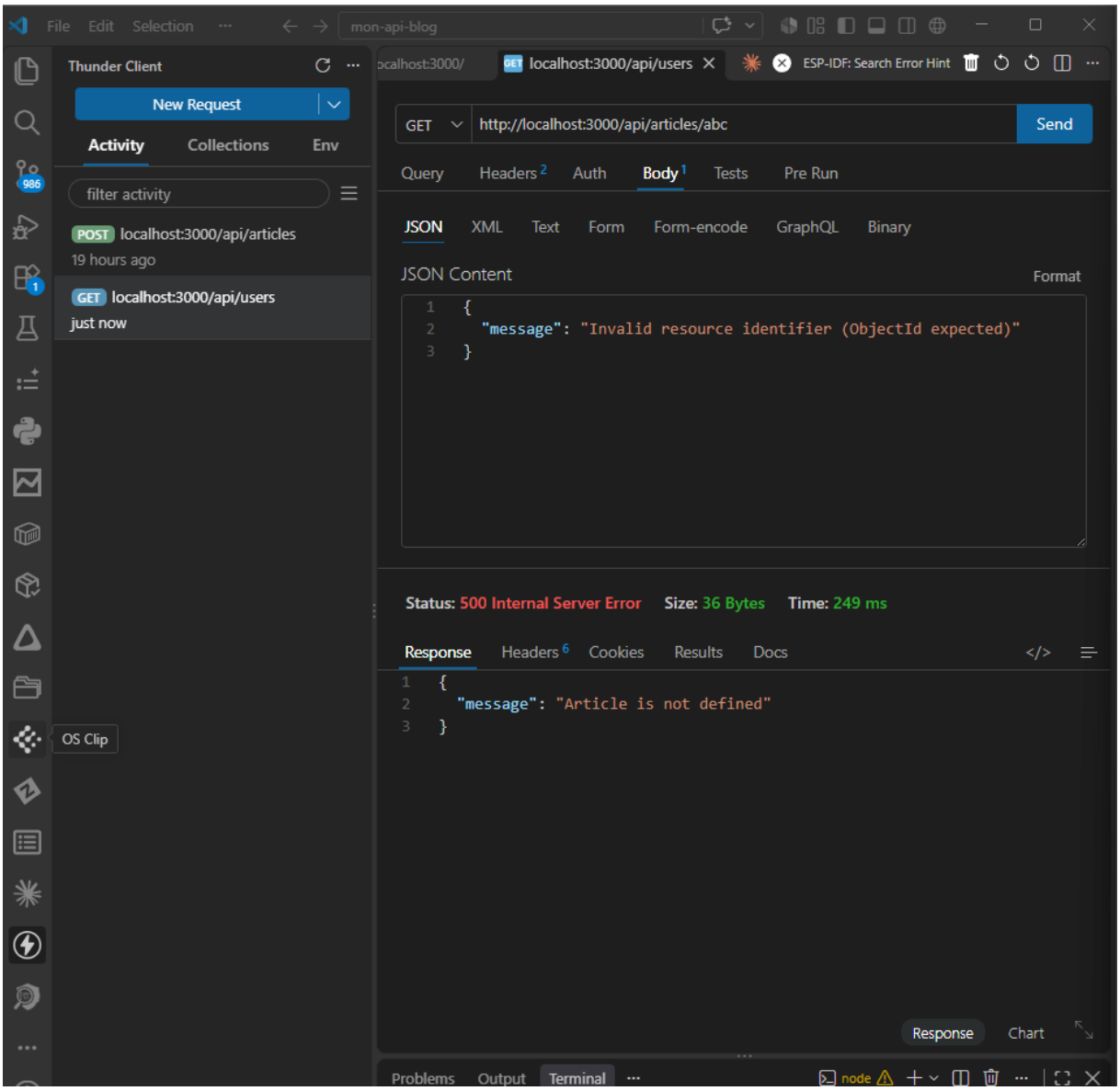
```
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non t
    }
    return res.status(200).json(user);
  } add a breakpoint err) {
    // Vérification si l'erreur est liée à un ID invalide (Cast
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expecte
      });
    }
    // Autre type d'erreur serveur
    return res.status(500).json({ message: err.message });
  }
};
```

2. Check specifically if the caught error matches err.name === 'CastError'.

3.If so, respond immediately with status 400 Bad Request and the following JSON message:

```
const getUserById = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouvé' });
    }
    return res.status(200).json(user);
  } catch (err) {
    // Vérification si l'erreur est liée à un ID invalide (CastError)
    if (err.name === 'CastError') {
      return res.status(400).json({
        message: 'Invalid resource identifier (ObjectId expected)'
      });
    }
    // Autre type d'erreur serveur                    any
    return res.status(500).json({ message: err.message });
  }
};
```

4. Test with Postman by sending a GET request to /api/articles/abc and verify that you receive a clean 400 response instead of 500.



2. Les captures d’écran de l’interface MongoDB Atlas montrant vos deux collections (users et articles) avec des documents insérés.

